Telethon

Telethon работает с API telegram client, что позволяет нам работать с группами/чатами в Telegram как пользователь. Другие библиотеки чаще всего работают с API telegram bot и имеют соответствующий функционал.

Установка

`pip install telethon` или `pip3 install telethon`

Импорт

```
from telethon import TelegramClient, events, sync, connection
```

Основные методы Telethon

В библиотеке **Telethon**, а точнее в ее документации, почти все объекты обозначаются словом **entity**. Для работы многих методов и запросов требуются сущности **entity**. **Entity** - это не какой-то определенный объект библиотеки **Telethon**, а скорее слово, которое можно перевести как объект, *сущность, организация, существо, вещь*. Например, когда вам нужно отправить запрос, вы отправляете сущность **entity**, когда получаете объект, - это тоже сущность **entity**. Есть множество вещей, которые работают как сущность **entity**, - `имена пользователей, номера телефонов, ссылка на чат, ссылки-приглашения, идентификаторы`. Замените для себя слово **entity** на слово объект, смысл не меняется.

Авторизация

- `client.start(entity)` - запускает клиент (подключается и авторизуется при необходимости), создаёт файл `*.session`;
- `client.send_code_request(entity)` - отправляет код Telegram, необходимый для входа на указанный номер телефона;
- `client.sign_in(entity)` - авторизуется в Telegram под существующей учетной записью пользователя или бота;
- `client.qr_login(entity)` - инициирует процедуру входа в систему QR;
- `client.log_out(entity)` - выходит из Telegram и удаляет текущий `*.session` - файл.

База

- `client.connect(entity)` - подключается к Телеграму;
- `client.disconnect(entity)` - отключается от Telegram;
- `client.is_connected(entity)` - возвращает `True`, если пользователь подключился;
- `client.disconnected(entity)` - свойство с а `Future`, которое разрешается при отключении;
- `client.loop(entity)` - свойство с `asyncio-` циклом событий, используемым этим клиентом;
- `client.set_proxy(entity)` - изменяет прокси, который будет использоваться при следующем (повторном) подключении.

Основные методы Telethon

Сообщения

- `client.send_message(entity)` - отправляет сообщение указанному пользователю, чату или каналу;
- `client.edit_message(entity)` - редактирует данное сообщение, чтобы изменить его текст или медиа;
- `client.delete_messages(entity)` - удаляет указанные сообщения, опционально «для всех»;
- `client.forward_messages(entity)` - пересылает данные сообщения указанному объекту;
- `client.iter_messages(entity)` - итератор по сообщениям для данного чата;
- `client.get_messages(entity)` - то же , что и `iter_messages()` , но вместо этого возвращает `TotalList` ;
- `client.pin_message(entity)` - закрепляет сообщение в чате;
- `client.unpin_message(entity)` - открепляет сообщение в чате;
- `client.send_read_acknowledge(entity)` - отмечает сообщения как прочитанные и при необходимости удаляет упоминания.

Загрузки

- `client.send_file(entity)` - отправляет сообщение с данным файлом указанному объекту;
- `client.upload_file(entity)` - загружает файл на серверы Telegram, не отправляя его.

Скачивания

- `client.download_media(entity)` - загружает указанный носитель из объекта сообщения;
- `client.download_profile_photo(entity)` - загружает фото профиля данного пользователя, чата или канала;
- `client.download_file(entity)` - низкоуровневый метод загрузки файлов из их исходного местоположения;
- `client.iter_download(entity)` - итерирует загрузку файла, получая фрагменты файла.

Основные методы Telethon

Диалоги

- `client.iter_dialogs(entity)` - итератор над диалогами (открытые разговоры/подписанные каналы);
- `client.get_dialogs(entity)` - то же , что и `iter_dialogs()` , но вместо этого возвращает `TotalList` ;
- `client.edit_folder(entity)` - редактирует папку, используемую одним или несколькими диалоговыми окнами для их архивирования;
- `client.iter_drafts(entity)` - итератор над черновиками сообщений;
- `client.get_drafts(entity)` - то же , что и `iter_drafts()` , но вместо этого возвращает список;
- `client.delete_dialog(entity)` - удаляет диалог (покидает чат или канал);
- `client.conversation(entity)` -создает объект `Conversation` с заданным объектом.

Пользователи

- `client.get_me(entity)` - получает «О себе», текущего пользователя, вошедшего в систему;
- `client.is_bot(entity)` - возврат `True` , если вошедший пользователь является ботом, `False` в противном случае;
- `client.is_user_authorized(entity)` - возвращает `True` , если пользователь авторизован (вошел в систему);
- `client.get_entity(entity)` - превращает данный объект в допустимого пользователя(`entity`) **Telegram** , чат или канал;
- `client.get_input_entity(entity)` - превращает данный объект в версию входного объекта;
- `client.get_peer_id(entity)` - получает идентификатор для данной сущности.

Чаты

- `client.iter_participants(entity)` - итератор по участникам, принадлежащим указанному чату;
- `client.get_participants(entity)` - то же , что и `iter_participants()` , но вместо этого возвращает `TotalList` ;
- `client.kick_participant(entity)` - выкидывает пользователя из чата;
- `client.iter_admin_log(entity)` - итератор над журналом администратора для указанного канала;
- `client.get_admin_log(entity)` - то же , что и `iter_admin_log()` но вместо этого возвращает `list` ;
- `client.iter_profile_photos(entity)` - итератор по фотографиям профиля пользователя или фотографиям чата;
- `client.get_profile_photos(entity)` - то же , что и `iter_profile_photos()` , но вместо этого возвращает `TotalList` ;
- `client.edit_admin(entity)` - редактирует права администратора для кого-либо в чате;
- `client.edit_permissions(entity)` - редактирует ограничения пользователя в чате;
- `client.get_permissions(entity)` - извлекает разрешения пользователя в определенном чате или канале или получает ограниченные права по умолчанию для чата или канала;
- `client.get_stats(entity)` - извлекает статистику из заданной мегагруппы или широковещательного канала;
- `client.action(entity)` - возвращает объект диспетчера контекста для представления «действия чата».

Парсинг сообщений из группы с Telethon

Для парсинга сообщений из группы или паблика **Telegram** используется два основных метода библиотеки **Telethon**. Первый метод - это `.get_messages()`, который возвращает указанное количество сообщений чата. Второй метод - это - `.iter_messages()`, который совершает итерации по каждому сообщению отдельно. У каждого из этих методов есть свои плюсы и минусы. Рассмотрим эти два метода более подробно.

Метод `client.get_messages()` - возвращает `TotalList` (**<class 'telethon.helpers.TotalList'>**), это список с дополнительными total свойством, который очень сильно похож на именованный кортеж (*namedtuple*), взаимодействие с ним происходит по тому же принципу, что и с `TotalList`.

Метод `client.iter_messages()` - итерируется по каждому сообщению чата отдельно, удобен, если мы хотим обрабатывать сообщения на лету.

- **user_id** - чтобы извлечь **ID** пользователя, отправившего сообщение, необходимо к объекту сообщения применить способ

  `message.from_id.user_id`;

- **date** - для того, чтобы извлечь дату отправки сообщения, используется способ `message.date`;
- **message** - чтобы извлечь текст сообщения, применяется два способа, `message.text` и `message.message`;
- **media** - используется для определения, присутствуют ли медиафайлы в сообщении или нет, как извлекать медиафайлы из сообщения мы поговорим далее;
- Доступ с остальным полям осуществляется подобным образом, с этим проблем у вас возникнуть не должно.

Обратите внимание на один важный момент. В объекте **Message**, хранится **user_id** в цифровом формате, а нам чаще всего необходимо знать его **username**, если он у него имеется. Чтобы конвертировать **user_id** в **username** используется метод `.get_entity(ID)`. Функция `.get_entity(ID)` возвращает объект **User** со всеми соответствующими этому объекту полями.

Выполнение функции `.get_entity(ID)` вернет объект, у которого имеются все необходимые поля, нам осталось извлечь из них **username.**

```
id_user =
message.from_id.user_id
print(client.get_entity(id_use
r).username)
```

Функции .iter_messages() и .get_messages()

`.get_messages()` и `.iter_messages()` принимают одинаковые аргументы:

- `client.get_messages(chat\group\dialog)` - первый обязательный аргумент, ссылка на телеграмм чат\паблик\диалог;
- `client.get_messages(limit)` - количество сообщений, которые необходимо собрать из группы\паблика\диалога, если `limit` не установлен, будет получено первое сообщение в чате;
- `client.get_messages(offset_date=datetime(2022, 6, 1))` - принимает объект `datatime` находит все сообщения ранее указанной даты, формат даты(год, месяц, день);
- `client.get_messages(offset_id(id_message))` - будут извлечены те сообщения, которые предшествуют сообщению с указанным `id_message`;
- `client.get_messages(max_id=30)` - будут исключены те сообщения, которые имеют более высокий идентификатор;
- `client.get_messages(min_id=30)` - будут исключены те сообщения, которые имеют более старый идентификатор;
- `client.get_messages(search='Искомый текст')` - строка, которая будет использоваться в качестве поискового запроса;
- `client.get_messages(from_user="@user")` -будут возвращены сообщения только этого пользователя. Имя можно указать как **username**, **id_user**, **last_name**, **first_name**;
- `client.get_message(reverse=True)` - Если установлено значение **True**, сообщения будут возвращаться в обратном порядке (от самого старого к самому новому, а не от нового к самому старому). Это также означает, что значения параметров **offset_id** и **offset_date** меняются местами;
- `client.get_messages(filter=InputMessagesFilterPhotos)` - фильтр используется для возврата сообщения. Например, данный фильтр вернет первое сообщение содержащее фото. Для работы с подобными фильтрами необходимо их импортировать `from telethon.tl.types import InputMessagesFilterPhotos`.

Функции .iter_messages() и .get_messages()

- Другие используемые фильтры (у каждого фильтра в списке, свой одноимённый импорт, по аналогии с `InputMessagesFilterPhotos`):
  - `filter=InputMessagesFilterChatPhotos` - вернет первое сообщение, содержащее фото;
  - `filter=InputMessagesFilterContacts` - вернет первое сообщение, содержащее контакт пользователя;
  - `filter=InputMessagesFilterDocument` - вернет первое сообщение, содержащее документ(pdf, docx, txt т т.д.);
  - `filter=InputMessagesFilterEmpty` - вернет первое сообщение, если его отправитель пуст, скорее всего это работает если пользователь был забанен в группе, а его сообщения не удалены;
  - `filter=InputMessagesFilterGeo` - вернет первое сообщение, в котором есть геометка;
  - `filter=InputMessagesFilterGif` - вернет первое сообщение, в котором содержится **GIF**;
  - `filter=InputMessagesFilterMusic` - вернет первое сообщение, если в нём содержится музыкальный файл;
  - `filter=InputMessagesFilterMyMentions` - вернет первое сообщение, в котором есть упоминания учетной записи, с которой происходит поиск;
  - `filter=InputMessagesFilterPhoneCalls` - вернет информацию об аудио-звонке;
  - `filter=InputMessagesFilterPhotoVideo` - вернет информацию об видео-звонке;
  - `filter=InputMessagesFilterPhotos` - используется для фильтрации результатов при поиске сообщений, чтобы получить только сообщения, содержащие фотографии;
  - `filter=InputMessagesFilterPinned` - вернет первое закреплённое сообщение;
  - `filter=InputMessagesFilterRoundVideo` - вернет первое сообщение, в котором содержится круглое видео;
  - `filter=InputMessagesFilterRoundVoice` - вернет первое сообщение, в котором содержится голосовое сообщение;
  - `filter=InputMessagesFilterUrl` - вернет первое сообщение, в котором содержится ссылка;
  - `filter=InputMessagesFilterVideo` - вернет первое сообщение, в котором содержится видео;
  - `filter=InputMessagesFilterVoice` - вернет первое сообщение, в котором содержится голосовое сообщение.

Отправляем результат парсинга в Telegram

```
client.send_message(en
tity)
```

```
from telethon import TelegramClient, events, sync, connection
r_api = 1********8
r_hash = 'b**********************b'

with TelegramClient('my', r_api, r_hash, system_version="4.10.5 beta x64") as
client:   client.send_message('@USERNAME', 'test')
```

Сообщение подобным образом можно отправить не только конкретному пользователю, но и чату\группе, если учетная запись, с которой вы отправляете сообщение, подписана на этот чат\группу.

Отправляем результат парсинга в Telegram

Метод `.send_message(entity)` принимает следующие аргументы:

- **entity='username'** - указывает на пользователя\чат\группу, куда необходимо отправить сообщение, если не указать пользователя, сообщение будет отправлено в чат "Избранное";
- **message='text'** - сообщение для отправки или другой объект для отправки, Максимальная длина сообщения составляет 35 000 байт или 4 096 символов;
- **reply_to(int\Message)** - отвечать на сообщение или нет, если указан int, он должен быть идентификатором сообщения, на которое следует ответить;
- **link_preview(=True\False)** - должен ли быть включен предварительный просмотр ссылок;
- **file(file='C:/file.txt')** - отправляет сообщение с вложенным файлом, файл может быть текстовым, видео, аудио или документом и т.д.;
- **thumb(str\byte\file)** - дополнительный эскиз для отправленного документа;
- **force_document(True\False)** - отправлять ли данный файл как документ;
- **buttons(custom.Button)** - отправляет кнопку, которая будет отображаться после отправки сообщения(в данном курсе мы не рассматривали работу с кнопками, кому интересна данная тема, можете самостоятельно почитать в документации)
- **silent(True\False)** = должно ли сообщение уведомлять людей в широковещательном канале или нет. По умолчанию **False**, что означает, что он уведомит их. Установите значение **True**, чтобы изменить это поведение.
- **background(True\False)** - должно ли сообщение быть отправлено в фоновом режиме.
- **schedule(hints.DateLike=None**) - устанавливает планировщик для отправки сообщения в указанную дату\время. По умолчанию установлен **None**, что означает что сообщение будет отправлено немедленно;
- **comment_to(int\message)** - оставляет "комментарий" в широковещательном канале. **int**, идентификатор паблика\группы. **message**, сообщение для отправки.